28. ORM'ler (Prisma, TypeORM, SQLAlchemy) ve Gizli Tehlike: N+1 Problemi

Tek Bir Sayfa İçin Veritabanına 1000 Sorgu Gönderme Tuzağından Kurtulmak
Bölüm 5 • Zorluk Derecesi: Orta / İleri Seviye • Tahmini Okuma Süresi: 12 dk

---

> 🎙️ **NotebookLM & Audio Overview Podcast Notları**
>
> **Podcast Tartışma Sorusu:**
> *"ORM'ler geliştiricilerin hayatını kolaylaştıran harika araçlar mı, yoksa arka planda sessizce binlerce sorgu atan tehlikeli bir canavar mı?"*
>
> **Bölümden Çıkarılacak 3 Altın Cümle:**
> - 📌 ORM (Object-Relational Mapping), nesne yönelimli kod ile ilişkisel veritabanı arasında köprü kurar; ancak SQL bilme zorunluluğunu ortadan kaldırmaz. - 📌 N+1 problemi, bir liste çekildikten sonra her elemanın ilişkili verisini döngü içinde ayrı ayrı sorgulama hatasıdır (1 + N sorgu). - 📌 `include` (Eager Loading) veya `JOIN` kullanarak N+1 sorgu felaketini tek bir birleşik sorguya indirebilirsiniz.

📖 1. Giriş Hikayesi & Günlük Hayat Analojisi

Bir süpermarkete gittiğinizi ve 50 farklı ürün alacağınızı düşünün.

Kasaya geldiniz. Sepetteki her ürünü tek tek geçirip ödemek yerine; her bir ürün için ayrı ayrı otoparka gidip arabanıza bir ürün bırakıp, tekrar kasaya dönüp ikinci ürünü alıp, tekrar otoparka gidip gelseydiniz ne olurdu? 50 kez arabaya gidip gelirdiniz ve gününüz biterdi!

İşte **N+1 Sorgu Problemi** tam olarak budur: Geliştirici 100 kullanıcının siparişlerini listelemek ister. 1. Önce 100 kullanıcıyı çeker (1 sorgu). 2. Sonra `for` döngüsünün içinde her kullanıcının siparişini tek tek veritabanına sorar (100 ayrı sorgu!). Toplamda 101 sorgu atılır! Veritabanı sunucusu nefes alamaz hale gelir.

Oysa *"Tüm kullanıcıları ve ilişkili siparişlerini tek seferde getir"* derseniz arabanıza tüm poşetleri tek seferde yüklersiniz.

🧠 2. AI Çağında Yeni Zihniyet Modeli

N+1 Probleminin Tespiti ve Çözümü:

```
YAVAŞ (N+1 Sorgu):
1. Sorgu: SELECT * FROM users LIMIT 50;
2. Sorgu: SELECT * FROM orders WHERE user_id = 1;
3. Sorgu: SELECT * FROM orders WHERE user_id = 2;
...
51. Sorgu: SELECT * FROM orders WHERE user_id = 50;
_____________________________________________________

HIZLI (Eager Loading - Tek Sorgu):
SELECT users.*, orders.*
FROM users
LEFT JOIN orders ON orders.user_id = users.id
LIMIT 50;
```

🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt)

Aşağıdaki prompt şablonunu Cursor, Claude Code, ChatGPT veya Copilot ortamınızda doğrudan kullanabilirsiniz:

````
# ROL: ORM ve Veritabanı Optimizasyon Uzmanı
Aşağıdaki Prisma / TypeScript fonksiyonu veritabanına yüzlerce gereksiz sorgu
atmaktadır.

## YAVAŞ KOD:
```typescript
export async function getUsersWithPosts() {
  const users = await prisma.user.findMany({ take: 100 });
  const results = [];
  for (const user of users) {
    // N+1 tuzağı! Her kullanıcı için ayrı sorgu atılıyor!
    const posts = await prisma.post.findMany({ where: { authorId: user.id } });
    results.push({ ...user, posts });
  }
  return results;
}
````

GÖREV:

1. N+1 problemini açıkla ve arka planda neden 101 sorgu çalıştığını göster.
2. Prisma'nın `include` (Eager Loading) özelliğini kullanarak bu işlemi tek bir veritabanı turuna (round-trip) indirecek optimize kodu yaz.
````

````
---

## 💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi

#### ❌ Acemi / Kontrolsüz AI Çıktısı (Riskli):
```typescript
// N+1 Tuzağı: Döngü içinde await sorgusu çalıştırmak
export async function getArticlesWithCommentsNaive() {
  const articles = await prisma.article.findMany({ take: 50 });

  // 50 makale için 50 kez veritabanına gidip gelir (Toplam 51 sorgu!)
  return await Promise.all(articles.map(async (article) => {
    const comments = await prisma.comment.findMany({ where: { articleId:
article.id } });
    return { ...article, comments };
  }));
}
````

✅ **Kıdemli Mühendis Tarafından İyileştirilmiş Sürüm (Güvenli & Temiz):**

```typescript
// Eager Loading ile Optimize Edilmiş Sürüm: Tek veya 2 birleşik sorgu
export async function getArticlesWithCommentsOptimized() {
  // Prisma arka planda tek bir JOIN veya IN sorgusuyla çözer
  return await prisma.article.findMany({
    take: 50,
    include: {
      comments: {
        select: {
          id: true,
          content: true,
          createdAt: true
        },
        orderBy: { createdAt: 'desc' },
        take: 5 // Sadece en son 5 yorumu alarak payload'ı hafiflet
      }
    }
  });
}
```

💡 **Kod Analizi:** `include` bloğu sayesinde Prisma tüm makaleleri ve ilişkili yorumları tek bir SQL sorgusunda çeker. 51 veritabanı turu (round-trip) 1 tura iner; sayfa yüklenme süresi 1500 ms'den 20 ms'ye düşer.

⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı

- **Tuzak:** Döngü (for/map) içinde veritabanı sorgusu (`await db...`) çalıştırmak.
```

- **Çözüm:** Döngü içinde asla sorgu çalıştırmayın; veriyi döngüden önce `include` veya `IN (...)` ile topluca çekin.

- **Tuzak:** ORM loglarını (query logging) kapatıp arkada ne kadar sorgu çalıştığını takip etmemek.
  - **Çözüm:** Geliştirme ortamında ORM SQL loglarını (`log: ['query']`) mutlaka açık tutun.
- **Tuzak:** İhtiyaç duyulmayan tüm devasa kolonları (`SELECT *`) çekmek.
  - **Çözüm:** `select` anahtarı ile sadece sayfada gösterilecek alanları çekerek bellek tasarrufu yapın.

🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri

🎥 Ders Başlığı: *Ders 28: ORM Optimizasyonu - N+1 Sorgu Tehlikesini Yok Etmek*

- 🖥️ **Slayt 1:** ORM nedir ve yazılımcıları nasıl tembelleştirir?
- 🖥️ **Slayt 2:** N+1 Probleminin anatomisi: 1 sorgu yerine 101 sorgu atmak.
- 🖥️ **Slayt 3:** Eager Loading (include) vs Lazy Loading mantığı.
- 🖥️ **Slayt 4:** Canlı SQL Log İncelemesi: 50 sorguyu 1 sorguya indirmek.

Canlı Kodlama / Ekran Kaydı Senaryosu:
Prisma'da `log: ['query']` aktif edin. Önce döngü içinde Post çeken fonksiyonu çalıştırıp terminali dolduran onlarca SQL sorgusunu gösterin. Ardından `include` ekleyip terminalde tek bir SQL sorgusu aktığını gösterin.

🎯 7. Günün Uygulamalı Görevi & Mini Sınav

🛠️ Günün Görevi:

Projenizde ORM sorgu loglarını aktif edin. Bir sayfayı açtığınızda arka planda kaç adet SQL sorgusu tetiklendiğini terminalden sayın.

🧠 Bölüm Sonu Kontrol Testi:

Soru 1: Veritabanı geliştirmede 'N+1 Problemi' tam olarak neyi ifade eder? - A) Veritabanına 1 ekleme yapınca N tanesinin silinmesini - B) Ana kayıtları getirmek için 1 sorgu, ardından döngü içinde her bir kaydın ilişkili verisini getirmek için N adet ayrı sorgu çalıştırılması felaketini - C) Sadece Python'da geçerli bir matematik teoremini - D) Tabloların adlarının 1 artırılmasını > **Doğru Yanıt:** B > **Açıklama:** N+1 problemi, ilişkili verilerin toplu çekilmek yerine döngü içinde her satır için ayrı ayrı sorgulanarak veritabanının gereksiz yere boğulmasıdır.

Soru 2: Prisma veya TypeORM gibi ORM'lerde N+1 problemini çözmek için kullanılan temel yaklaşım hangisidir? - A) Eager Loading (`include` veya `relations` kullanarak ilişkili veriyi tek seferde çekmek)

- B) Veritabanını kapatıp açmak - C) Tüm verileri frontend'e aktarmak - D) Sadece string veri kullanmak > **Doğru Yanıt:** A > **Açıklama:** Eager Loading, ilişkili kayıtların ana sorguyla birlikte JOIN veya optimize toplu sorgularla tek seferde belleğe alınmasını sağlar.